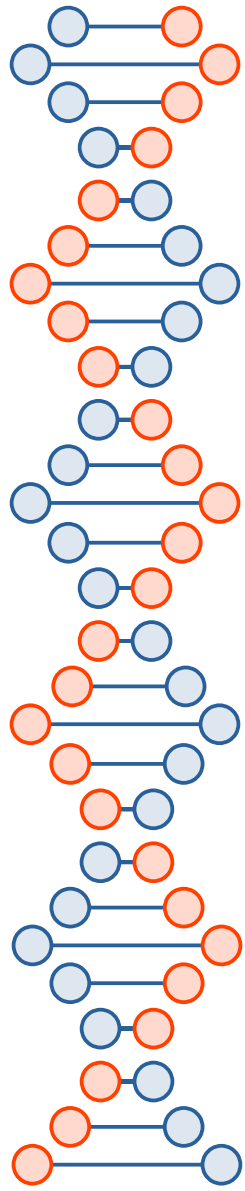


Arreglos Asociativos en Perl (Hashes)

Comparación con arreglos y referencias

By PhD. Josafá Pontes

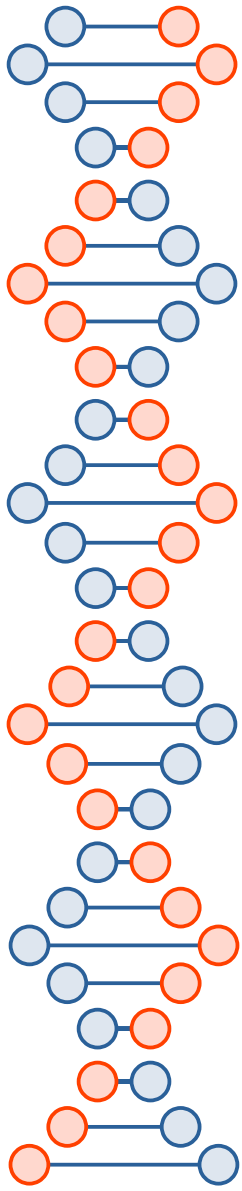


Objetivos

- Entender qué es un hash en Perl
- Compararlo con:
- Arreglos (@array)
- Referencias a arreglos (\$array_ref)
- Aprender cuándo usar cada estructura
- Ver ejemplos prácticos
- Practicar mediante ejercicio

Tipos de estructuras en Perl

- Escalar → \$x
- Arreglo (Array) → @arr
- Hash (Arreglo asociativo) → %hash
- Referencias → \$ref



Arreglos (@array)

- Colección ordenada de elementos indexados numéricamente.

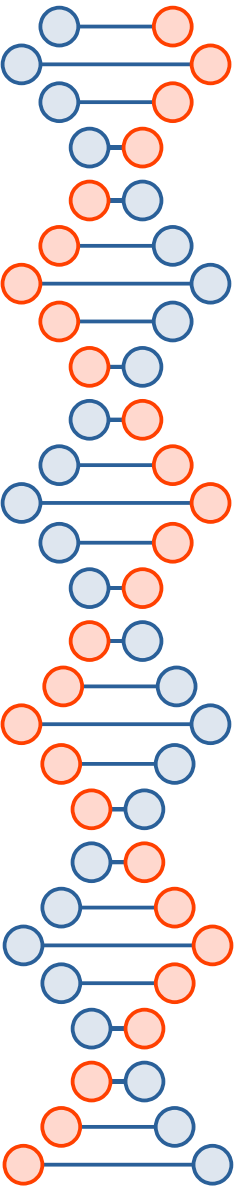
```
my @nums = (10, 20, 30);  
print $nums[0]; # 10;
```

Índices: 0, 1, 2...

Mantienen orden

Acceso por posición

→ No permite subgrupar los datos

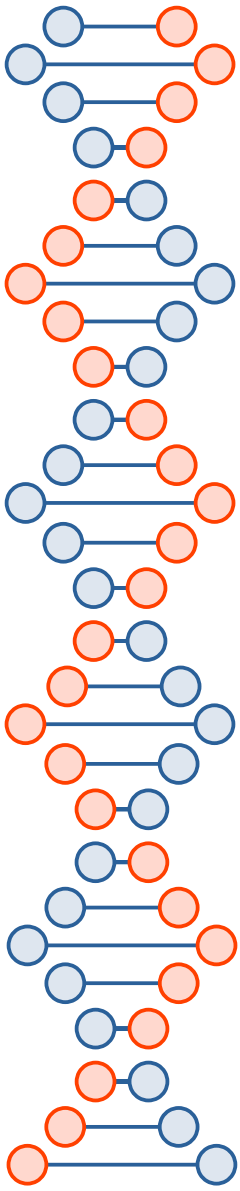


Referencias a arreglos

- Una referencia apunta a un arreglo.

```
my $arr_ref = [10, 20, 30];  
print $arr_ref->[1]; # 20
```

Permiten estructuras complejas (subagrupaciones)
Útiles para pasar datos a funciones



Hashes (%hash)

- Colección de pares clave → valor.

```
my %edad = ("Ana" => 25, "Luis" => 30);  
print $edad{"Ana"}; # 25
```

No usan índices numéricos

Acceso por clave

No garantizan orden

Comparación conceptual

Estructura	Tipo de acceso	Orden	Clave
Array (@)	Índice	Sí	Número
Ref Array (\$)	Índice	Sí	Número
Hash (%)	Clave	No	String

Sintaxis comparada

Array indexados

```
my @a = (1, 2, 3);
```

```
print $a[0]; 1
```

Ref Array indexados

```
my $r = [1, 2, 3];
```

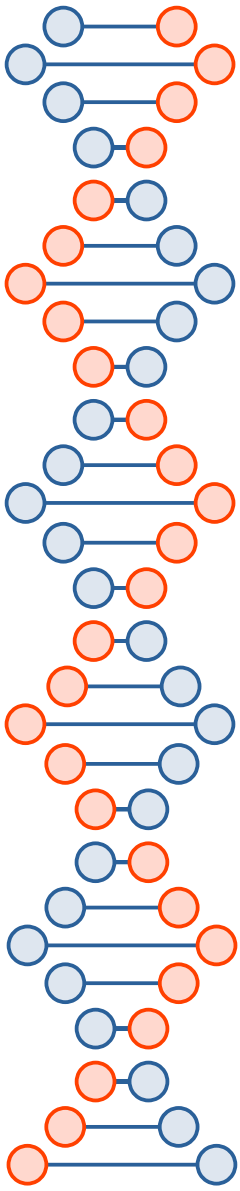
```
print $r->[0]; # desreferenciación de un elemento
```

```
print @$r; # desreferenciación de todo el array
```

Hash

```
my %h = (a => 1, b => 2);
```

```
print $h{"a"};
```

Ejemplo práctico

- Problema: Guardar notas de estudiantes

Con array:

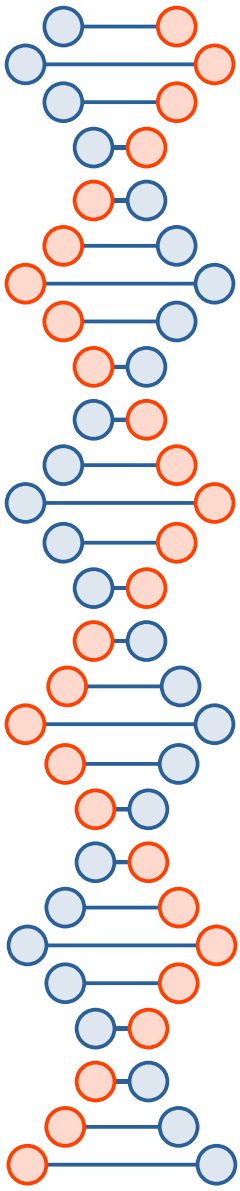
```
my @notas = (15, 18, 20);
```

No sabemos a quién pertenece cada nota

Con hash:

```
my %notas = ("Ana" => 15, "Luis" => 18, "Pedro" => 20);
```

Más claro y útil

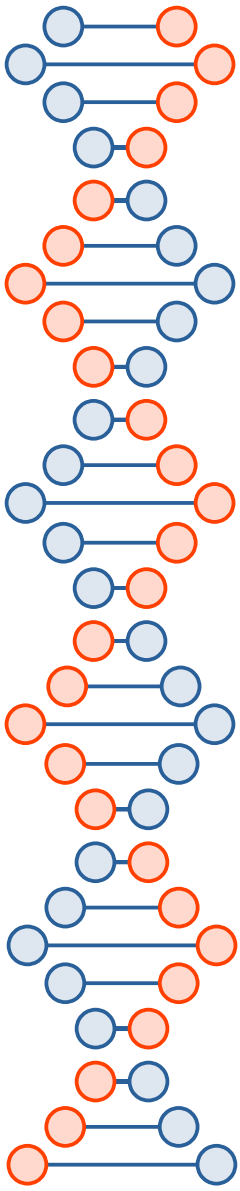


Iteración

- Array:
for my \$x (@nums) {
 printf "%s", \$x;
}
- Hash:
for my \$key (**keys** %edad) {
 printf "%s: %d\n", \$key, \$edad{\$key};
}

Estructuras combinadas

```
my %clase = (  
    "Ana" => [15, 18, 20],  
    "Luis" => [12, 14, 16]  
);  
  
print $clase{"Ana"}->[1]; # 18  
Hash de arrays  
Muy usado en data science
```



Errores comunes

- Confundir \$ y @
- Usar índice en hash:
\$hash[0]; # Incorrecto
\$hash{"0"}; # correcto
- Olvidar -> en referencias:
\$ref[0]; #
\$ref->[0]; #

¿Cuándo usar cada uno?

- Usa Array → datos ordenados
- Usa Hash → búsqueda rápida por clave
- Usa referencias → estructuras complejas o anidadas

Referencias a Hashes

- Una referencia a hash apunta a una estructura %hash.

```
my $h_ref = {  
    "Ana" => 25,  
    "Luis" => 30
```

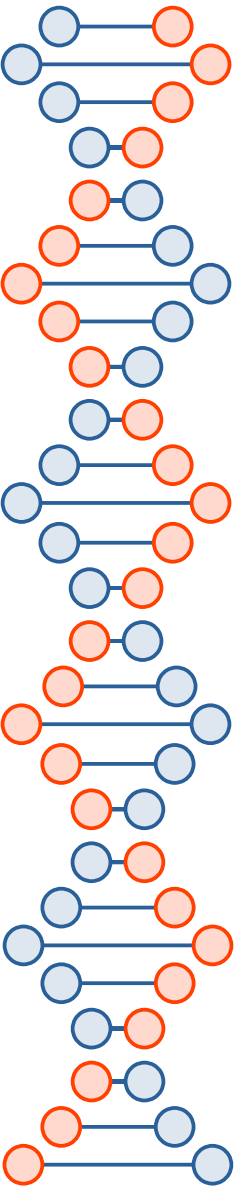
```
};
```

```
print $h_ref->{"Ana"}; # 25 Desreferenciar elemento
```

```
print dump %$h_ref; # Desreferenciar todo el hash
```

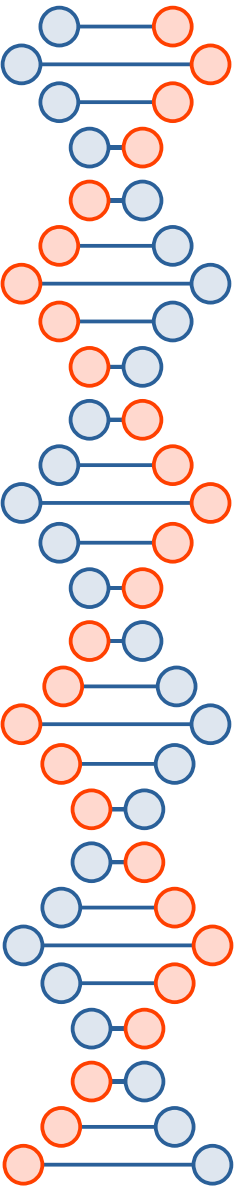
Similar a hash, pero usando \$

Necesita -> para acceder



Comparación: Ref Array vs Ref Hash

Característica	Ref Array (\$arr_ref)	Ref Hash (\$hash_ref)
Tipo	Lista ordenada	Pares clave → valor
Acceso	\$ref->[i]	\$ref->{key}
Orden	Sí	No garantizado
Uso típico	Agrupaciones de secuencias	Diccionarios y estructuras/jerarquía



Sintaxis lado a lado

Referencia a array

```
my $arr = [10, 20, 30];
```

```
print $arr->[1]; # 20
```

Referencia a hash

```
my $hash = { a => 1, b => 2 };
```

```
print $hash->{a}; # 1
```

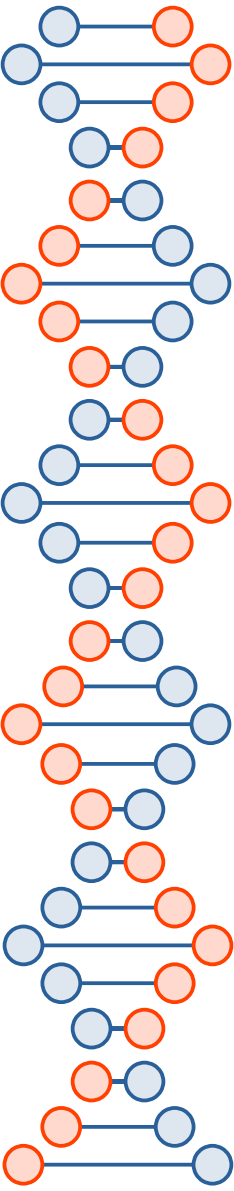

Estructuras combinadas

- Hash de arrays:

```
my $data = {"Ana" => [10, 20], "Luis" => [30, 40] };  
print $data->{"Ana"}->[0]; # 10
```

- Array de hashes:

```
my $data = [  
    { nombre => "Ana", edad => 25 },  
    { nombre => "Luis", edad => 30 }  
];  
print $data->[0]->{nombre}; # Ana
```



Errores comunes con referencias

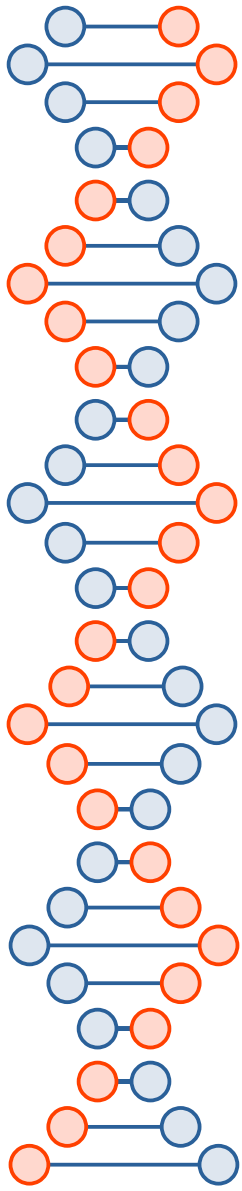
- Confundir tipos:
`$ref->{0};` # incorrecto si es array
`$ref->[0];` # incorrecto si es hash
- Correcto:
`$arr_ref->[0];` # array indexado
`$hash_ref->{key};` # array asociativo

Tabla resumen global

Estructura	Símbolo de declaración	Acceso	Orden	Toma de un elemento
Array	@	[i]	Sí	\$a[0]
Hash	%	{key}	No	\$h{"a"}
Ref Array	\$	->[i]	Sí	\$r->[0]
Ref Hash	\$	->{key}	No	\$r->{a}

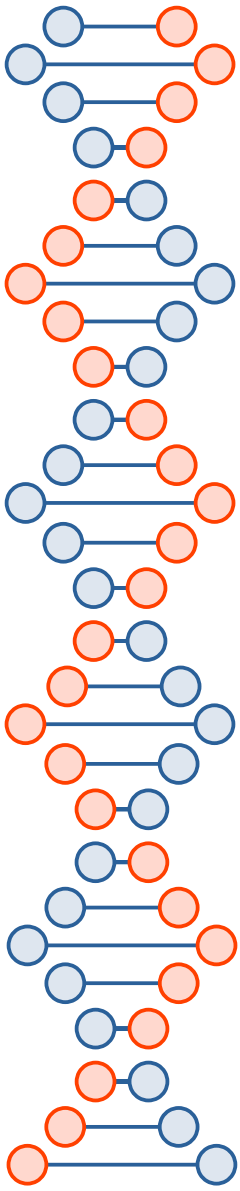
¿Por qué usar referencias?

- Evitan copiar datos grandes
- Permiten estructuras anidadas
- Necesarias para programación avanzada
- Facilitan paso de datos a funciones



Ejemplo completo

```
my $clase = {  
    "Ana" => {  
        edad => 20,  
        notas => [15, 18, 20]  
    }  
};  
  
print $clase->{"Ana"}->{notas}->[1]; # 18
```



Ejercicio

- Dada la siguiente estructura:

```
my $db = [  
    { nombre => "Ana", notas => [10, 20] },  
    { nombre => "Luis", notas => [15, 25] }  
];
```

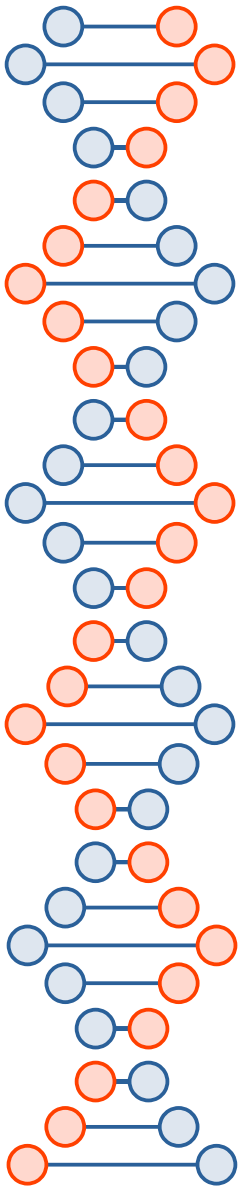
→ Tareas:

- Acceda a una nota
- Promedie las notas por estudiante

```
use List::Util qw(sum);
```
- Itere toda la estructura y subestructuras con 3 comandos for anidados

```
if (ref($arr_ref) eq 'ARRAY')
```

Output esperado



- Notas de Ana: [10, 20]
- Promedio de notas de Ana: 15.00
- Promedio de notas de Luis: 20.00
- Ana
- 10 20
- Luis
- 15 25